

● SESIÓN 8B — Inyección de Queries con @Query (JPQL + SQL nativo) + JOINS

"Cuando el método derivado no llega, escribimos la query."

■ Bloque 1 — Por qué ahora

Hasta ahora ya sabemos:

- CRUD con JpaRepository
- Paginación + filtros básicos (sesión 8)

Pero en la vida real aparecen necesidades como:

- JOIN con columnas concretas ("reporting")
- GROUP BY, agregaciones
- consultas optimizadas y específicas de MySQL
- datos planos (no entities) para frontend

📄 💡 Mensaje clave:

"DTO/proyección para resultados. Entity para persistencia."

Bloque 2 — Qué vamos a hacer (5 min)

Objetivo práctico

Crear endpoints de "consultas avanzadas":

1 JOIN 2 tablas (Book + Category)
→ listado "plano" con categoryName

2 JOIN 3 tablas (Book + Category + Publisher)
→ reporte más realista

Bloque 3 — Estructura del proyecto

com.cladsb1.booksdb

├─ controller
| ├─ BookDbController
| └─ BookQueryController

Bloque 4 — Qué archivos crear/modificar (lista clara)

Crear

- entity/Publisher.java
- repository/projection/BookWithCategoryView.java
- repository/projection/BookFullView.java
- service/BookQueryService.java
- controller/BookQueryController.java

Modificar

- entity/BookDb.java (añadir publisher si hacemos JOIN 3 tablas)
- repository/BookDbRepository.java (añadir métodos @Query)
- (opcional) PublisherRepository.java



Bloque 5 — Preparación del modelo (JOIN 3 tablas)

5.1 Entity Publisher (NUEVA)

entity/Publisher.java

```
package com.cladsb1.booksdb.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "publishers")
public class Publisher {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable=false, unique=true, length=80)
    private String name;

    @Column(nullable=false, length=40)
    private String country;

    protected Publisher() {}

    public Publisher(String name, String country) {
        this.name = name;
        this.country = country;
    }

    public Long getId()    { return id; }
    public String getName() { return name; }
    public String getCountry() { return country; }

    public void setName(String name)    { this.name = name; }
    public void setCountry(String country) { this.country = country; }
}
```

5.2 Modificar BookDb para añadir publisher (si no lo tienes)

entity/BookDb.java (fragmento)

```
@ManyToOne(optional = false, fetch = FetchType.LAZY)
@JoinColumn(name = "publisher_id", nullable = false)
private Publisher publisher;

public Publisher getPublisher() { return publisher; }
public void setPublisher(Publisher publisher) {
    this.publisher = publisher;
}
```

Bloque 6 — Projections (lo más didáctico para @Query)

6.1 JOIN 2 tablas: Book + Category

 repository/projection/BookWithCategoryView.java

```
package com.cladsb1.booksdb.repository.projection;

public interface BookWithCategoryView {
    Long getBookId();
    String getTitle();
    String getAuthor();
    String getCategoryName();
}
```

6.2 JOIN 3 tablas: Book + Category + Publisher

 repository/projection/BookFullView.java

```
package com.cladsb1.booksdb.repository.projection;

public interface BookFullView {
    Long getBookId();
    String getTitle();
    String getAuthor();
    String getCategoryName();
    String getPublisherName();
    String getPublisherCountry();
}
```

  Mensaje clave:

"Projection = respuesta plana optimizada (no entity)."



Bloque 7 — Inyección de Query en Repository

📁 repository/BookDbRepository.java (añadir estos métodos; repo debe estar bien tipado)

```
package com.cladsb1.booksdb.repository;

import com.cladsb1.booksdb.entity.BookDb;
import com.cladsb1.booksdb.repository.projection.BookFullView;
import com.cladsb1.booksdb.repository.projection.BookWithCategoryView;
import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param;
import java.util.List;

public interface BookDbRepository
    extends JpaRepository<BookDb, Long> {

    // -----
    // A) JOIN 2 tablas (SQL nativo)
    //   Devuelve projection (no entity)
    // -----
    @Query(value = """
        SELECT b.id    AS bookId,
               b.title AS title,
               b.author AS author,
               c.name   AS categoryName
        FROM books b
        JOIN categories c ON b.category_id = c.id
        WHERE (:category IS NULL
              OR LOWER(c.name) = LOWER(:category))
        ORDER BY b.title ASC
        """, nativeQuery = true)
    List<BookWithCategoryView> findBooksWithCategoryNative(
        @Param("category") String category
    );

    // -----
    // B) JOIN 3 tablas (SQL nativo)
    // -----
    @Query(value = """
        SELECT b.id    AS bookId,
               b.title AS title,
               b.author AS author,
               c.name   AS categoryName,
               p.name   AS publisherName,
               p.country AS publisherCountry
        FROM books b
        JOIN categories c ON b.category_id = c.id
        JOIN publishers p ON b.publisher_id = p.id
        WHERE (:category IS NULL
              OR LOWER(c.name) = LOWER(:category))
              AND (:country IS NULL
                  OR LOWER(p.country) = LOWER(:country))
        ORDER BY b.title ASC
        """, nativeQuery = true)
    List<BookFullView> findBooksReport3TablesNative(
        @Param("category") String category,
        @Param("country") String country
    );
}
```



✓ Observa la regla de oro:

Los alias AS bookId, AS categoryName... **deben coincidir** con getters de la projection.



Bloque 8 — Service (CON y SIN streams)

Aquí vamos a enseñar 2 estilos:

A) Devolviendo directamente Projection
(más simple)

B) Transformando Projection → DTO
(útil para contrato API)

Te dejo la opción **B** para poder mostrar "con/sin streams" claramente.

DTOs de respuesta (opcionales)

📁 dto/response/BookWithCategoryDTO.java

```
package com.cladsb1.booksdb.dto.response;

public record BookWithCategoryDTO(
    Long bookId,
    String title,
    String author,
    String categoryName
) {}
```

📁 dto/response/BookFullDTO.java

```
package com.cladsb1.booksdb.dto.response;

public record BookFullDTO(
    Long bookId,
    String title,
    String author,
    String categoryName,
    String publisherName,
    String publisherCountry
) {}
```



A) Service CON streams/lambda

service/BookQueryService.java

```
package com.cladsb1.booksdb.service;

import com.cladsb1.booksdb.dto.response.BookFullDTO;
import com.cladsb1.booksdb.dto.response.BookWithCategoryDTO;
import com.cladsb1.booksdb.repository.BookDbRepository;
import com.cladsb1.booksdb.repository.projection.BookFullView;
import com.cladsb1.booksdb.repository.projection.BookWithCategoryView;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class BookQueryService {

    private final BookDbRepository bookDbRepository;

    public BookQueryService(BookDbRepository bookDbRepository) {
        this.bookDbRepository = bookDbRepository;
    }

    public List<BookWithCategoryDTO> findBooksWithCategory(String category) {
        if (category != null && category.isBlank()) category = null;

        List<BookWithCategoryView> rows =
            bookDbRepository.findBooksWithCategoryNative(category);

        //
```




B) Service SIN streams/lambda

(Solo cambia el mapping de listas)

```
public List<BookWithCategoryDTO> findBooksWithCategory(String category) {  
    if (category != null && category.isBlank()) category = null;  
  
    List<BookWithCategoryView> rows =  
        bookDbRepository.findBooksWithCategoryNative(category);  
  
    //
```


Bloque 9 — Controller (endpoints de consultas avanzadas)

 controller/BookQueryController.java

```
package com.cladsb1.booksdb.controller;

import com.cladsb1.booksdb.dto.response.BookFullDTO;
import com.cladsb1.booksdb.dto.response.BookWithCategoryDTO;
import com.cladsb1.booksdb.service.BookQueryService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/queries")
public class BookQueryController {

    private final BookQueryService bookQueryService;

    public BookQueryController(BookQueryService bookQueryService) {
        this.bookQueryService = bookQueryService;
    }

    /**
     * JOIN 2 tablas:
     * /api/queries/books-with-category
     * /api/queries/books-with-category?category=programming
     */
    @GetMapping("/books-with-category")
    public ResponseEntity<List<BookWithCategoryDTO>> booksWithCategory(
        @RequestParam(required = false) String category
    ) {
        return ResponseEntity.ok(
            bookQueryService.findBooksWithCategory(category));
    }

    /**
     * JOIN 3 tablas (reporte):
     * /api/queries/books-report
     * /api/queries/books-report?category=programming
     * /api/queries/books-report?country=spain
     * /api/queries/books-report?category=programming&country=spain
     */
    @GetMapping("/books-report")
    public ResponseEntity<List<BookFullDTO>> report3Tables(
        @RequestParam(required = false) String category,
        @RequestParam(required = false) String country
    ) {
        return ResponseEntity.ok(
            bookQueryService.report3Tables(category, country));
    }
}
```



Bloque 10 — Qué probar en Swagger

01

GET /api/queries/books-with-category

02

GET /api/queries/books-with-category?
category=programming

03

GET /api/queries/books-report

04

GET /api/queries/books-report?country=spain

05

GET /api/queries/books-report?
category=programming&country=spain

Bloque 11 — Errores típicos

✗ Repository sin genéricos
→ todo se convierte en
Object

`extends JpaRepository<BookDb,
Long>` obligatorio

✗ Projection sin alias
correcto (AS ...) → getters
devuelven null / error

✗ Usar nombres Java en
SQL (hay que usar columnas
reales)

✗ Intentar mapear
resultado parcial a Entity

Para reporting →
Projection/DTO

✗ `publisher_id` no existe en
tabla `books` → el join 3
tablas falla (lógico)

Bloque 12 — Cierre conceptual



Regla final para clase:

Derived queries / Specifications (Sesión 9) para búsquedas normales. **@Query native** para reporting y joins potentes.

Derived Queries

Búsquedas normales y sencillas

@Query native

Reporting, JOINS potentes y agregaciones